MODULE 10 · DOMAIN 1 - KUBERNETES FUNDAMENTALS

kubectl fluency

The commands, flags and habits that exam questions assume you know.

DOMAIN 1 of 4 EXAM WEIGHT 44% MODULE 10 of 35

admin@sthithapragna.com

Every kubectl command is a verb, a resource type, a name, and some flags.

The grammar of a command

```
$ kubectl get deployment booking-api -n quayside -o yaml

#         verb           name                    output
#             type                    namespace

$ kubectl describe pod web-7d9f6c-x2k4p
$ kubectl delete svc web -n quayside
$ kubectl get pods,svc -A
```

1 Verb, type, name, then flags. The order of flags does not matter.

8 Several types at once, across all namespaces.

ON THE EXAM

Types accept singular, plural or a short name: pod, pods and po all work.

Short names worth knowing

SHORT	KIND	SHORT	KIND
po	pods	deploy	deployments
svc	services	rs	replicasets
ns	namespaces	sts	statefulsets
cm	configmaps	ds	daemonsets
pvc	persistentvolumeclaims	pv	persistentvolumes
sa	serviceaccounts	no	nodes

ON THE EXAM

kubectl api-resources lists every short name. Exam commands often use them.

kubeconfig: how kubectl knows where to go

- **A file,** by default ~/.kube/config, or any path in the KUBECONFIG variable.
- **clusters:** each API server's URL and CA certificate.
- **users:** credentials: a client certificate, a token, or an exec plugin.
- **contexts:** a cluster plus a user plus a default namespace. current-context picks one.

> ON THE EXAM
>
> A context is not a cluster. It is a named pairing of cluster, user and namespace.

Switching contexts

```
$ kubectl config get-contexts
CURRENT   NAME             CLUSTER        AUTHINFO     NAMESPACE
*         quayside-stage   stage-cluster  ci-deployer  quayside
          quayside-prod    prod-cluster   ci-deployer  quayside

$ kubectl config use-context quayside-prod
$ kubectl config set-context --current --namespace=payments
$ kubectl get pods --context quayside-stage
```

3 The asterisk marks the current context.

6 Switch every later command to production.

7 Change the current context's default namespace.

8 Target a context for one command only.

create versus apply

KUBECTL CREATE -F

Imperative

- Makes a new object
- Fails if it already exists
- You choose the operation
- Good for one-off objects

KUBECTL APPLY -F

Declarative

- Creates it, or updates it to match
- Safe to run again and again
- Works on files and whole folders
- The basis of GitOps

THE TIEBREAKER

apply is idempotent: run it twice and nothing changes the second time. create is not.

Generating manifests instead of writing them

```
1 $ kubectl create deployment web --image=quayside/web:5.2.0 \
2     --replicas=3 --dry-run=client -o yaml > web.yaml
3
4 $ kubectl expose deployment web --port=80 --target-port=8080
5
6 $ kubectl run debug --image=busybox:1.37 --restart=Never -it --rm -- sh
```

1-2 --dry-run=client sends nothing; -o yaml prints the manifest to a file.

4 expose creates a Service for existing Pods.

6 run creates a single Pod. Here: a throwaway shell, deleted on exit.

kubectl run: just a Pod

- **kubectl run creates one bare Pod.** Not a Deployment, not a Job.
- **Old material says otherwise:** years ago, run created Deployments. It no longer does.
- **A bare Pod is not healed.** Fine for a quick test, wrong for anything that must stay up.
- **For a Deployment,** use kubectl create deployment, or apply a manifest.

> ON THE EXAM
>
> If an option claims kubectl run creates a Deployment or a ReplicaSet, it is out of date.

Getting output in the shape you need

FLAG	GIVES YOU
`-o wide`	Extra columns, including the node and Pod IP
`-o yaml`, `-o json`	The full object, spec and status
`-o jsonpath='{.status.podIP}'`	One field, for scripts
`-o custom-columns=NAME:.metadata.name,NODE:.spec.nodeName`	Your own table
`-w`, `--watch`	Keep streaming changes
`--sort-by=.metadata.creationTimestamp`	Rows in a chosen order

ON THE EXAM

'Which node is each Pod on?' is get pods -o wide.

get versus describe

KUBECTL GET

Lists and data

- One line per object
- Machine-friendly -o formats
- Good for scripts and quick checks
- Shows STATUS, not reasons

KUBECTL DESCRIBE

Detail and events

- Everything about one object
- Human-readable
- Recent Events at the bottom
- Shows why, not just what

THE TIEBREAKER

'Why is it Pending?' is describe. The Events section holds the answer.

Changing live objects

COMMAND	DOES
`kubectl edit deploy/web`	Opens the live object in an editor; saving applies it
`kubectl patch deploy/web -p '...'`	Changes just the fields you give
`kubectl set image | env | resources`	Targeted changes to a Pod template
`kubectl scale deploy/web --replicas=5`	Changes the replica count
`kubectl label | annotate`	Adds or changes metadata
`kubectl diff -f web.yaml`	Previews what apply would change, without changing it

ON THE EXAM

diff before apply shows exactly what will change. It is the safest habit in this table.

Three small changes

```
$ kubectl patch deployment web -p '{"spec":{"replicas":5}}'
$ kubectl set env deployment/booking-api LOG_LEVEL=debug
$ kubectl diff -f web.yaml
-  replicas: 5
+  replicas: 3
```

1 A merge patch: only replicas changes.

2 Changes the template, so it triggers a rollout.

3-5 The file says 3; the cluster says 5. apply would scale back down.

Imperative edits and the source of truth

- **edit, patch, scale and set** change the cluster, not your files.
- **The next apply from Git** puts the file's values back.
- **A GitOps tool** will revert them even sooner, as drift.
- **Emergencies are fine,** then copy the change into Git straight away.

> ON THE EXAM
>
> If the file and the cluster disagree, the next apply makes the cluster match the file.

Deleting, safely

COMMAND	RESULT
`kubectl delete -f web.yaml`	Deletes whatever the file defines
`kubectl delete pod web-7d9f6c-x2k4p`	Its controller creates a replacement
`kubectl delete deploy web`	The Deployment, its ReplicaSets and Pods
`kubectl delete ns staging`	Everything in the namespace
`--grace-period=0 --force`	Skips graceful shutdown. Last resort only

ON THE EXAM

Deleting a managed Pod is a restart, not a removal. To remove it, delete its owner.

Inspection commands, covered later

COMMAND	FOR	MODULE
`kubectl logs`	A container's output	25
`kubectl exec -it pod -- sh`	A shell inside a container	25
`kubectl port-forward`	Reach a Pod from your laptop	25
`kubectl debug`	An ephemeral debug container	25
`kubectl top pods`	CPU and memory use, via metrics-server	26
`kubectl auth can-i`	Check a permission	17

ON THE EXAM

kubectl top needs metrics-server. Without it, top reports that metrics are unavailable.

Versions, plugins and completion

- **Version skew:** kubectl is supported within one minor version of the API server.
- A 1.37 kubectl works with 1.36, 1.37 and 1.38 control planes.
- **Plugins:** any executable named kubectl-something becomes kubectl something. Krew installs them.
- **Completion:** kubectl completion bash or zsh enables tab completion.

> ON THE EXAM
>
> Old kubectl against a much newer cluster is unsupported. Keep it within one minor version.

> An engineer needs a starting manifest for a new Deployment called web, using image quayside/web:5.2.0 with 3 replicas.
>
> Nothing may be created in the cluster yet; the file will be reviewed in Git first.

Which command produces the manifest?

A kubectl run web --image=quayside/web:5.2.0

B kubectl get deployment web -o yaml > web.yaml

C kubectl explain deployment --recursive > web.yaml

D kubectl create deploy web … --dry-run=client -o yaml

◆ PAUSE AND DECIDE

Which command produces the manifest?

A kubectl run web --image=quayside/web:5.2.0

Creates a real bare Pod, not a Deployment manifest.

B kubectl get deployment web -o yaml > web.yaml

The Deployment does not exist yet, so there is nothing to get.

C kubectl explain deployment --recursive > web.yaml

Saves field documentation, not a manifest.

D kubectl create deploy web ... --dry-run=client -o yaml

Builds the object locally and prints it. Nothing is sent.

THE SIGNAL

'Generate YAML, create nothing' is --dry-run=client -o yaml.

> During an incident, the on-call engineer needs to see which node each booking API Pod is running on, and each Pod's IP address.
>
> They want one quick command, with one line per Pod.

Which command gives exactly that?

A kubectl describe deployment booking-api

B kubectl get pods -l app=booking-api -o wide

C kubectl top pods -l app=booking-api

D kubectl get deployment booking-api -o yaml

◆ PAUSE AND DECIDE

Which command gives exactly that?

A kubectl describe deployment booking-api
Describes the Deployment, not where each Pod landed.

B kubectl get pods -l app=booking-api -o wide
One line per Pod, with NODE and IP columns.

C kubectl top pods -l app=booking-api
Shows CPU and memory use, not nodes or IPs.

D kubectl get deployment booking-api -o yaml
The Deployment's spec and status. Pods are not listed.

THE SIGNAL

Node and IP per Pod: get pods -o wide.

> An engineer deleted the web Deployment in production, believing they were connected to staging. Both contexts use the same namespace name.
>
> The team wants a habit that would most directly have prevented this.

Which habit would most directly have prevented it?

A Always add --force when deleting, to finish quickly

B Use kubectl run for changes instead of delete

C Check the current context, or pass --context explicitly

D Keep production and staging in one shared context

◆ PAUSE AND DECIDE

Which habit would most directly have prevented it?

A Always add --force when deleting, to finish quickly

Makes a mistake faster and harsher, not less likely.

B Use kubectl run for changes instead of delete

run creates Pods; it has nothing to do with the target cluster.

C Check the current context, or pass --context explicitly

The command went to the wrong cluster. Context decides that.

D Keep production and staging in one shared context

A context points at one cluster; this only adds confusion.

THE SIGNAL

Wrong cluster means wrong context. Check it before anything destructive.

> The web Deployment is managed from Git, with a pipeline running kubectl apply on every merge. The file sets replicas: 3.
>
> During an incident, on-call ran kubectl scale to 10. The next morning, after an unrelated merge, there are 3 replicas again.

Why did the count return to 3?

A The pipeline's apply made the cluster match the file again

B kubectl scale changes expire after about twelve hours

C The ReplicaSet restored its original replica count

D etcd rolled back the change during the overnight backup

◆ PAUSE AND DECIDE

Why did the count return to 3?

A The pipeline's apply made the cluster match the file again

The file still says 3, and apply makes the cluster match it.

B kubectl scale changes expire after about twelve hours

No kubectl change expires. It lasts until something changes it.

C The ReplicaSet restored its original replica count

A ReplicaSet follows its Deployment; it keeps no memory of old counts.

D etcd rolled back the change during the overnight backup

Backups copy state; they never roll it back.

THE SIGNAL

Imperative change, then an apply from Git: the file wins. Put emergency changes into Git.

Module 10 in five lines

- **Verb, type, name, flags.** Short names like po, svc, deploy, ns, cm.
- A **context** is cluster + user + namespace, stored in **kubeconfig**.
- **apply is idempotent;** create fails if it exists. **run** makes a bare Pod.
- **--dry-run=client -o yaml** generates manifests; **-o wide** shows node and IP.
- **describe for why**; imperative edits are undone by the next **apply**.

You can drive a cluster. Next: **administering one**, from kubeadm to upgrades and draining nodes.